

Aula 38 - Gabarito da Atividade Prática

Aqui está o gabarito completo estruturado para correção da atividade. Ele contém o código PHP refatorado seguindo as boas práticas e o modelo de relatório técnico esperado.

Resumo das Violações Inseridas:

- **Quebra do SRP (Single Responsibility Principle):** A classe `PaymentManager` não tem apenas um motivo para mudar. Ela precisará ser alterada se as regras de validação de CPF mudarem, se o formato do recibo em texto for atualizado ou se a forma de enviar SMS for substituída por e-mail ou WhatsApp.
- **Quebra do OCP (Open/Closed Principle):** A cascata de `if/else` para calcular a taxa da transação impede que o sistema seja estendido sem modificação. A introdução de um pagamento via "Criptomoeda", por exemplo, forçaria a abertura e alteração direta deste arquivo.
- **Quebra do DIP (Dependency Inversion Principle):** Embora o foco da atividade seja SRP e OCP, o código instiga instintivamente a aplicação do DIP, já que escreve em disco e "envia SMS" de forma concreta e acoplada, em vez de depender de contratos/interfaces (`ReciboInterface` ou `NotificadorInterface`).

1. Código-Fonte Refatorado (PaymentManager.php)

Nesta solução, isolamos as responsabilidades (SRP) em classes distintas e utilizamos interfaces e polimorfismo para lidar com as regras de cálculo (OCP).

PHP

```
<?php
```

```
// =====  
// 1. Abstrações (Interfaces)  
// =====
```

```
interface MetodoPagamentoInterface
```

```
{  
    public function calcularTaxa(float $valor): float;  
    public function getNome(): string;  
}
```

```
interface ReciboInterface
```

```
{  
    public function gerar(string $nomeCliente, string $cpf, float $valorOriginal, float $taxa, float $valorTotal):  
void;  
}
```

```
interface NotificadorInterface
```

```
{  
    public function notificar(string $destinatario, string $mensagem): void;  
}
```

```
// =====
```

```
// 2. Implementações de Pagamento (OCP)
// =====
```

```
class CartaoCredito implements MetodoPagamentoInterface
{
    public function calcularTaxa(float $valor): float
    {
        return $valor * 0.05; // 5% de taxa
    }

    public function getNome(): string
    {
        return 'CARTAO_CREDITO';
    }
}
```

```
class Boleto implements MetodoPagamentoInterface
{
    public function calcularTaxa(float $valor): float
    {
        return 2.50; // Taxa fixa
    }

    public function getNome(): string
    {
        return 'BOLETO';
    }
}
```

```
class Pix implements MetodoPagamentoInterface
{
    public function calcularTaxa(float $valor): float
    {
        return 0.00; // Sem taxa
    }

    public function getNome(): string
    {
        return 'PIX';
    }
}
```

```
// =====
// 3. Serviços Especialistas (SRP)
// =====
```

```

class ValidadorCliente
{
    public static function validar(string $nomeCliente, string $cpf): void
    {
        if (empty($nomeCliente) || empty($cpf)) {
            throw new Exception("Erro: Dados do cliente são obrigatórios.");
        }

        if (strlen($cpf) !== 11) {
            throw new Exception("Erro: CPF inválido.");
        }
    }
}

```

```

class GeradorReciboArquivo implements ReciboInterface
{
    public function gerar(string $nomeCliente, string $cpf, float $valorOriginal, float $taxa, float $valorTotal):
void
    {
        $conteudo = "--- RECIBO DE PAGAMENTO ---\n";
        $conteudo .= "Cliente: {$nomeCliente} (CPF: {$cpf})\n";
        $conteudo .= "Valor Original: R$ " . number_format($valorOriginal, 2, ',', '.') . "\n";
        $conteudo .= "Taxa Aplicada: R$ " . number_format($taxa, 2, ',', '.') . "\n";
        $conteudo .= "Total Pago: R$ " . number_format($valorTotal, 2, ',', '.') . "\n";

        $nomeArquivo = "recibo_" . time() . ".txt";
        file_put_contents($nomeArquivo, $conteudo);
        echo "Recibo salvo em: {$nomeArquivo}\n";
    }
}

```

```

class NotificadorSms implements NotificadorInterface
{
    public function notificar(string $destinatario, string $mensagem): void
    {
        echo "Enviando SMS para o cliente (CPF: {$destinatario}): '{$mensagem}'\n";
    }
}

```

```

// =====
// 4. Classe Principal (Orquestradora)
// =====

```

```

class PaymentManager
{
    private ReciboInterface $geradorRecibo;

```

```

private NotificadorInterface $notificador;

// Utilizando o Princípio da Inversão de Dependência (DIP) para injetar serviços
public function __construct(ReciboInterface $geradorRecibo, NotificadorInterface $notificador)
{
    $this->geradorRecibo = $geradorRecibo;
    $this->notificador = $notificador;
}

public function processarPagamento(string $nomeCliente, string $cpf, float $valor,
MetodoPagamentoInterface $metodoPagamento): bool
{
    // 1. Validação delegada
    ValidadorCliente::validar($nomeCliente, $cpf);

    // 2. Cálculo de taxa delegado ao polimorfismo
    $taxa = $metodoPagamento->calcularTaxa($valor);
    $valorTotal = $valor + $taxa;

    echo "Processando pagamento de R$ " . number_format($valorTotal, 2, ',', '.') . " via " .
$metodoPagamento->getNome() . "...\\n";

    // 3. Geração de recibo delegada
    $this->geradorRecibo->gerar($nomeCliente, $cpf, $valor, $taxa, $valorTotal);

    // 4. Notificação delegada
    $mensagem = "Pagamento de R$ " . number_format($valorTotal, 2, ',', '.') . " aprovado com sucesso.";
    $this->notificador->notificar($cpf, $mensagem);

    return true;
}
}

// =====
// Exemplo de uso (Refatorado)
// =====

try {
    // Injeção de dependências
    $geradorRecibo = new GeradorReciboArquivo();
    $notificador = new NotificadorSms();

    $gerenciador = new PaymentManager($geradorRecibo, $notificador);

    $gerenciador->processarPagamento("João da Silva", "12345678901", 150.00, new CartaoCredito());
    echo "\\n";
}

```

```
$gerenciador->processarPagamento("Maria Souza", "10987654321", 300.00, new Pix());  
} catch (Exception $e) {  
    echo $e->getMessage() . "\n";  
}  
  
?>
```

2. Gabarito do Relatório Técnico (README.md)

Mapeamento de Violações no Código Legado

- **Violações do SRP (Responsabilidade Única):** A classe PaymentManager original operava como uma "ferramenta multifuncional". Ela acumulava a validação de formato do CPF, o cálculo matemático das tarifas, a manipulação direta do sistema de arquivos (fopen/fwrite) e a lógica de disparo de mensagens SMS. Consequentemente, qualquer mudança em uma dessas regras poderia quebrar o processamento de pagamentos.
- **Violações do OCP (Aberto/Fechado):** O método processarPagamento possuía um bloco de if/else rígido para definir as taxas baseadas em strings ('CARTAO_CREDITO', 'BOLETO', 'PIX'). O código violava a regra de que entidades devem estar abertas para extensão e fechadas para modificação. Adicionar um novo método de pagamento exigiria a alteração obrigatória desta classe central.

Justificativa da Solução (Como o SOLID foi aplicado)

- **Aplicação do SRP:** Para garantir que cada classe tenha "apenas um motivo para mudar", a lógica foi fragmentada:
 - Criou-se a classe ValidadorCliente para centralizar as regras de validação.
 - Criou-se a classe GeradorReciboArquivo (e sua respectiva interface) para isolar a escrita de arquivos.
 - Criou-se a classe NotificadorSms para lidar especificamente com comunicações.
 - Desta forma, separamos as lógicas de negócio das lógicas de comunicação e persistência.
- **Aplicação do OCP:** Substituímos o longo bloco de condicionais (if/else) por polimorfismo e interfaces.
 - Foi criada a interface MetodoPagamentoInterface contendo o contrato calcularTaxa().
 - Cada método de pagamento transformou-se em uma classe especialista (CartaoCredito, Boleto, Pix).
 - A classe PaymentManager agora depende unicamente da interface. É possível plugar novos meios de pagamento sem jamais alterar o gerenciador de pagamentos, blindando a classe contra quebras futuras.